

HDOC DAY 41

PROBLEM

Write a menu-driven C program that reads two paragraphs. Use separate user-defined functions for each operation. Use call by reference by passing pointers wherever a function modifies a string or returns multiple results. Perform the following tasks:

1. Read two paragraphs from the user.
 - `void readParagraph(char paragraph[], size_t size);`
2. For each paragraph, identify and display the longest and shortest words.
 - Hint: Use `strtok()`, `strlen()`, and `strcpy()`.
 - `void findLongestAndShortestWords(const char paragraph[], char longestWord[], char shortestWord[]);`
3. Remove repeated consecutive spaces from each paragraph and display the modified paragraph.
 - Hint: Use `isspace()` and a character-by-character scan.
 - `void removeRepeatedSpaces(const char paragraph[], char modifiedParagraph[]);`
4. Convert each paragraph to title case and display the modified paragraph.
 - Hint: Use `isalpha()`, `isspace()`, `tolower()`, and `toupper()`.
 - `void convertToTitleCase(const char paragraph[], char titleCaseParagraph[]);`
5. Find and display the frequency of every distinct word in each paragraph.
 - Words must be compared without considering differences in letter case.
 - Hint: Use `strtok()`, `strcmp()`, `strcpy()`, and `tolower()`.
 - `void findWordFrequencies(const char paragraph[]);`
6. Exit

CODE

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <ctype.h>
```

```
#define SIZE 1000
```

```
#define MAX_WORDS 500
```

```
#define WORD_SIZE 100
```

```
// Function declarations
```

```
void readParagraph(char paragraph[], size_t size);
```

```
void findLongestAndShortestWords(  
    const char paragraph[],  
    char longestWord[],  
    char shortestWord[]  
);
```

```
void removeRepeatedSpaces(  
    const char paragraph[],  
    char modifiedParagraph[]  
);
```

```
void convertToTitleCase(  
    const char paragraph[],  
    char titleCaseParagraph[]  
);
```

```
void findWordFrequencies(const char paragraph[]);
```

```
// Function to read a paragraph
```

```
void readParagraph(char paragraph[], size_t size)
{
    fgets(paragraph, size, stdin);

    // Remove newline character
    paragraph[strcspn(paragraph, "\n")] = '\0';
}
```

// Function to find longest and shortest words

```
void findLongestAndShortestWords(
    const char paragraph[],
    char longestWord[],
    char shortestWord[])
{
    char temp[SIZE];
    char *word;

    strcpy(temp, paragraph);

    longestWord[0] = '\0';
    shortestWord[0] = '\0';

    word = strtok(temp, " \\t\\n.,!?;:");

    while (word != NULL)
    {
        if (longestWord[0] == '\0' ||
            strlen(word) > strlen(longestWord))
```

```

    {
        strcpy(longestWord, word);
    }

    if (shortestWord[0] == '\0' ||
        strlen(word) < strlen(shortestWord))
    {
        strcpy(shortestWord, word);
    }

    word = strtok(NULL, " \\t\\n.,!?:;");
}
}

```

// Function to remove repeated consecutive spaces

```

void removeRepeatedSpaces(
    const char paragraph[],
    char modifiedParagraph[])
{
    int i = 0, j = 0;
    int previousSpace = 0;

    while (paragraph[i] != '\0')
    {
        if (isspace((unsigned char)paragraph[i]))
        {
            if (!previousSpace)
            {

```

```

        modifiedParagraph[j++] = paragraph[i];
    }

    previousSpace = 1;
}
else
{
    modifiedParagraph[j++] = paragraph[i];
    previousSpace = 0;
}

    i++;
}

    modifiedParagraph[j] = '\0';
}

// Function to convert paragraph into title case
void convertToTitleCase(
    const char paragraph[],
    char titleCaseParagraph[])
{
    int i = 0;
    int newWord = 1;

    while (paragraph[i] != '\0')
    {
        if (isalpha((unsigned char)paragraph[i]))

```

```

{
    if (newWord)
    {
        titleCaseParagraph[i] =
            toupper((unsigned char)paragraph[i]);
        newWord = 0;
    }
    else
    {
        titleCaseParagraph[i] =
            tolower((unsigned char)paragraph[i]);
    }
}
else
{
    titleCaseParagraph[i] = paragraph[i];

    if (isspace((unsigned char)paragraph[i]))
    {
        newWord = 1;
    }
}

i++;
}

titleCaseParagraph[i] = '\0';
}

```

```

// Function to find frequency of every distinct word
void findWordFrequencies(const char paragraph[])
{
    char temp[SIZE];
    char words[MAX_WORDS][WORD_SIZE];
    int frequency[MAX_WORDS] = {0};

    int count = 0;
    int i, j, found;

    char *word;

    strcpy(temp, paragraph);

    word = strtok(temp, " \\t\\n.,!?:;");

    while (word != NULL)
    {
        // Convert word to lowercase
        for (i = 0; word[i] != '\\0'; i++)
        {
            word[i] = tolower((unsigned char)word[i]);
        }

        found = 0;

        // Check whether word already exists

```

```

for (i = 0; i < count; i++)
{
    if (strcmp(words[i], word) == 0)
    {
        frequency[i]++;
        found = 1;
        break;
    }
}

// Store new word
if (!found && count < MAX_WORDS &&
    strlen(word) < WORD_SIZE)
{
    strcpy(words[count], word);
    frequency[count] = 1;
    count++;
}

word = strtok(NULL, " \\t\\n.,!?:;");
}

printf("\nWord Frequencies:\n");

for (i = 0; i < count; i++)
{
    printf("%-20s : %d\n", words[i], frequency[i]);
}

```

```
}
```

```
// Main function
```

```
int main()
```

```
{
```

```
    char paragraph1[SIZE], paragraph2[SIZE];
```

```
    char longest[WORD_SIZE], shortest[WORD_SIZE];
```

```
    char modified[SIZE];
```

```
    char titleCase[SIZE];
```

```
    int choice, p;
```

```
    char *paragraph;
```

```
    printf("Enter the first paragraph:\n");
```

```
    readParagraph(paragraph1, SIZE);
```

```
    printf("\nEnter the second paragraph:\n");
```

```
    readParagraph(paragraph2, SIZE);
```

```
    do
```

```
    {
```

```
        printf("\n===== MENU =====\n");
```

```
        printf("1. Find Longest and Shortest Words\n");
```

```
        printf("2. Remove Repeated Consecutive Spaces\n");
```

```
        printf("3. Convert Paragraph to Title Case\n");
```

```
        printf("4. Find Frequency of Each Word\n");
```

```
printf("5. Exit\n");  
printf("=====\n");
```

```
printf("Enter your choice: ");  
scanf("%d", &choice);
```

```
if (choice >= 1 && choice <= 4)  
{  
    printf("\nSelect Paragraph:\n");  
    printf("1. First Paragraph\n");  
    printf("2. Second Paragraph\n");  
    printf("Enter your choice: ");  
    scanf("%d", &p);
```

```
    if (p == 1)  
        paragraph = paragraph1;  
    else if (p == 2)  
        paragraph = paragraph2;  
    else  
    {  
        printf("Invalid paragraph choice!\n");  
        continue;  
    }  
}
```

```
switch (choice)  
{  
    case 1:
```

```
findLongestAndShortestWords(  
    paragraph, longest, shortest);  
  
if (longest[0] == '\0')  
{  
    printf("\nParagraph contains no words.\n");  
}  
else  
{  
    printf("\nLongest Word : %s\n", longest);  
    printf("Shortest Word : %s\n", shortest);  
}  
break;
```

case 2:

```
removeRepeatedSpaces(paragraph, modified);  
  
printf("\nModified Paragraph:\n%s\n", modified);  
  
// Update original paragraph  
strcpy(paragraph, modified);  
break;
```

case 3:

```
convertToTitleCase(paragraph, titleCase);  
  
printf("\nTitle Case Paragraph:\n%s\n",  
    titleCase);
```

```
        // Update original paragraph
        strcpy(paragraph, titleCase);

        break;

    case 4:

        findWordFrequencies(paragraph);

        break;

    case 5:

        printf("\nExiting Program...\n");

        break;

    default:

        printf("\nInvalid choice! Try again.\n");

    }

} while (choice != 5);

return 0;

}
```